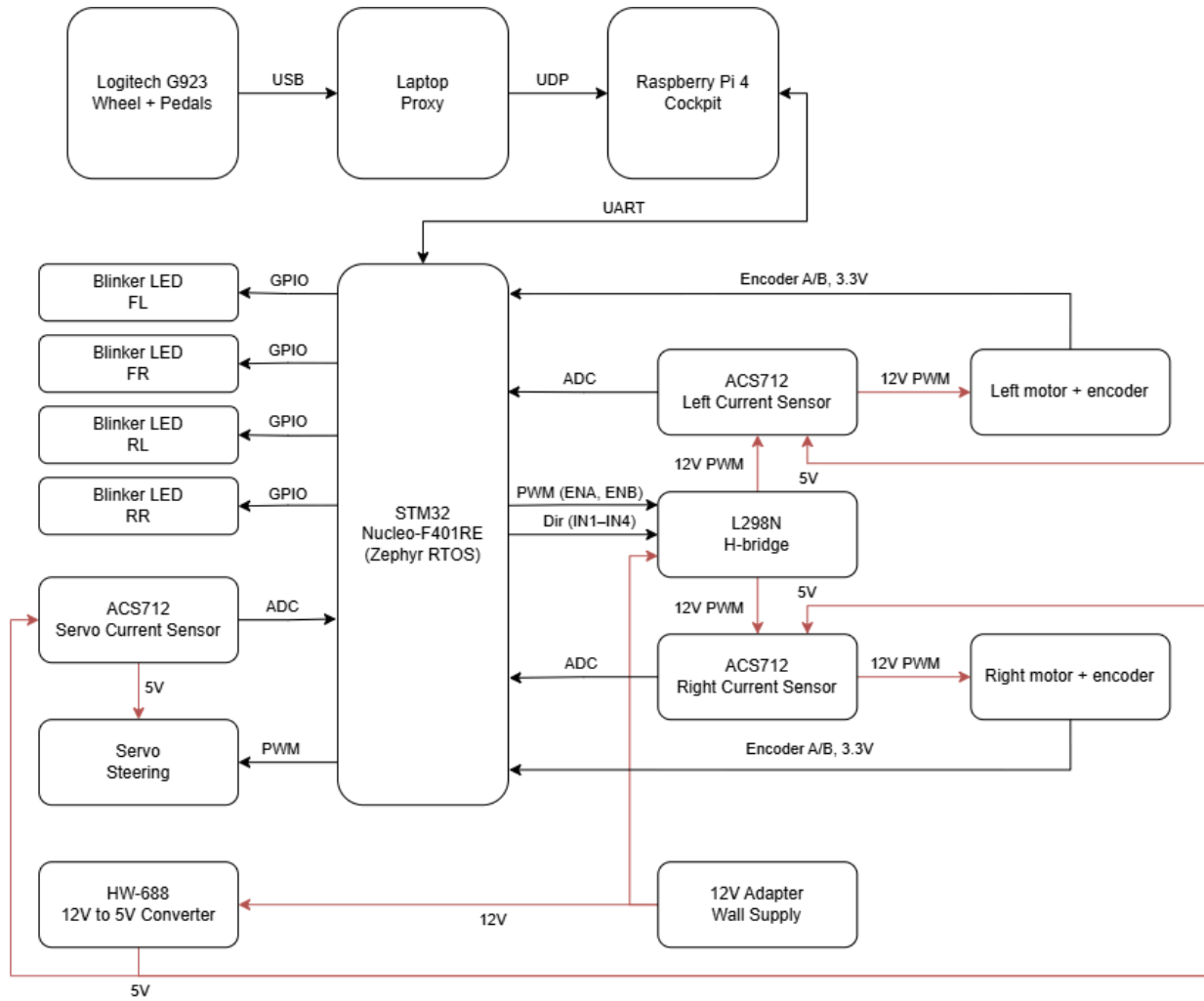
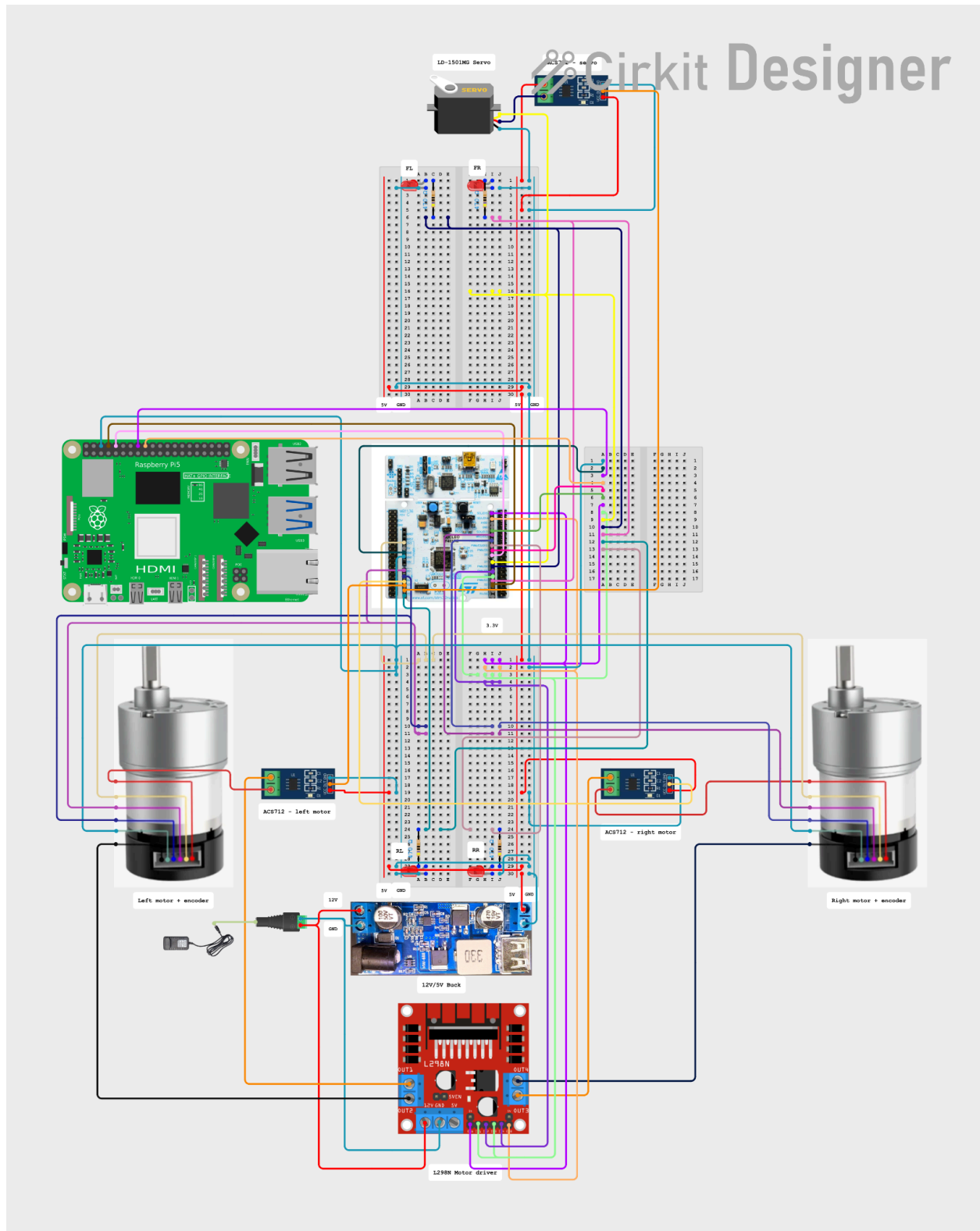


Circuit Documentation

Lab 2: Sensor/Actuator Bring-up.



*All grounds tied to a common ground rail



1. Circuit Summary

The circuit is a single-MCU version of the drive-by-wire car. It combines a Raspberry Pi 5, an STM32 NUCLEO-F401RE, an L298N dual H-bridge, two encoder geared motors (rear drive), an LD-1501MG steering servo, three ACS712 current sensors, four blinker LEDs, an HW-688 5 V buck converter, a 12 V barrel-jack input and a test-point board.

- The 12 V wall adapter powers the L298N (motor supply) and the HW-688.
- The HW-688 produces a 5 V rail that powers the servo (through its current sensor) and all three ACS712 sensors.
- The Nucleo is powered from the laptop over USB; the Raspberry Pi uses its own official supply. Neither powers the other.
- All grounds meet at one main ground rail on the rear board.
- The STM32 drives the L298N and servo, reads both encoders and all three current sensors, and drives the four blinkers.
- The Raspberry Pi talks to the STM32 over a 3.3 V UART (STM32 USART1).
- Every timing-relevant signal is brought out to a 13-row test-point board.

Only the Arduino headers of the Nucleo are used. D0/D1 are left unconnected because on the Nucleo they are tied to the ST-LINK virtual COM port by default (used for debug output over USB).

2. Power and Ground

Rail	Source	Feeds
12 V	Wall adapter via barrel-jack screw terminals	L298N 12V terminal, HW-688 VIN+ (split at the barrel terminals)
5 V (car)	HW-688 5 V output, rear board 5 V rail	Servo (through ACS712 servo sensor), VCC of all three ACS712s; one 5 V + GND pair runs to the front board
3.3 V	Nucleo +3V3 pin (CN6)	Both motor encoders (sensor +)
Nucleo 5 V	Laptop USB (JP5 on U5V)	Nucleo only. +5V, E5V and VIN pins are intentionally unconnected
Pi 5 V	Official Raspberry Pi USB-C supply	Raspberry Pi only
L298N logic 5 V	L298N on-board regulator (5V-EN jumper on)	L298N logic only. 5V terminal left unconnected

Ground. The barrel-jack negative goes directly into the L298N GND screw terminal so motor return current stays in screw terminals, off the breadboard. One wire each runs to the main ground rail from:

- L298N GND terminal and HW-688 GND
- Nucleo GND (CN6)
- Raspberry Pi GND (pin 6)
- Front board GND row (servo, servo sensor, front LEDs)
- Motor sensor GNDs, rear LEDs, encoder grounds
- Test-point board rows 1 and 2

12 V is never placed on a breadboard.

3. STM32 NUCLEO-F401RE Pin Map

Header	Pin	Function	Connected to	Test point
D2	PA10	USART1 RX	Pi GPIO14 (TX, pin 8)	-
D8	PA9	USART1 TX	Pi GPIO15 (RX, pin 10)	-
D15	PB8	TIM4 CH3 PWM	L298N ENA (left motor)	Row 7 PWM_OUT
D14	PB9	TIM4 CH4 PWM	L298N ENB (right motor)	-
D4	PB5	GPIO out	L298N IN1 and IN3	Row 8 DIR_A
D5	PB4	GPIO out	L298N IN2 and IN4	-
D7	PA8	TIM1 CH1 PWM	Servo signal	Row 9 SRV
A0	PA0	TIM2 CH1 encoder	Left encoder A	-
A1	PA1	TIM2 CH2 encoder	Left encoder B	-
D12	PA6	TIM3 CH1 encoder	Right encoder A	-
D11	PA7	TIM3 CH2 encoder	Right encoder B	-
A3	PB0	ADC1 IN8	Left motor ACS712 OUT	-
A2	PA4	ADC1 IN4	Right motor ACS712 OUT	-
A4	PC1	ADC1 IN11	Servo ACS712 OUT	-
D6	PB10	GPIO out	FL LED (via 470 Ω)	Row 10 FL
D10	PB6	GPIO out	FR LED (via 470 Ω)	Row 11 FR
A5	PC0	GPIO out	RL LED (via 470 Ω)	Row 12 RL
D3	PB3	GPIO out	RR LED (via 470 Ω)	Row 13 RR
D9	PC7	GPIO out	Test point only	Row 5 CMD_RX
D13	PA5	GPIO out	Test point only (on-board LD2 follows it)	Row 6 PWM_SET
+3V3	-	3.3 V out	Encoder + (both motors)	-
GND	-	Ground	Main ground rail	-

Signals that go both to a part and to a test point are split on a breadboard row first, since each Arduino header socket holds one jumper.

4. Components

Raspberry Pi 5

Powered from its own supply. All GPIO is 3.3 V and not 5 V tolerant.

Pi pin	Signal	Connected to
8 (GPIO14)	UART TX	STM32 D2 (PA10, RX)
10 (GPIO15)	UART RX	STM32 D8 (PA9, TX)
6	GND	Main ground rail
16 (GPIO23)	UDP_RX test point	Test-point row 3
18 (GPIO24)	CMD_TX test point	Test-point row 4

The serial login console must be disabled on the Pi so the GPIO UART is free.

L298N Dual H-Bridge

ENA and ENB jumper caps removed (PWM from the STM32). 5V-EN jumper left on.

L298N terminal	Connected to
12V	Barrel-jack + (12 V)
GND	Barrel-jack –, and one wire to the main ground rail
5V	Not connected (on-board regulator output)
ENA	STM32 D15 (PB8)
IN1, IN3	STM32 D4 (PB5)
IN2, IN4	STM32 D5 (PB4)
ENB	STM32 D14 (PB9)
OUT1	Left motor ACS712 IP+
OUT2	Left motor red wire
OUT3	Right motor ACS712 IP+
OUT4	Right motor black wire

Drive Motors (JGB37-520, 1:30, with encoder)

The two motors drive the rear wheels. Each 6-wire cable splits: the two power wires go to the L298N side and the four encoder wires go to the Nucleo side, routed apart from each other.

Wire	Left motor	Right motor
Motor power + (red)	L298N OUT2	Right ACS712 IP–
Motor power – (black)	Left ACS712 IP–	L298N OUT4

Wire	Left motor	Right motor
Encoder +	Nucleo +3V3	Nucleo +3V3
Encoder -	Ground	Ground
Encoder A	A0 (PA0)	D12 (PA6)
Encoder B	A1 (PA1)	D11 (PA7)

The left motor's leads are swapped relative to the right on purpose (see Design Notes).

Steering Servo (LD-1501MG)

- Red (V+): servo ACS712 IP-
- Brown/black (GND): front board GND row
- Signal: STM32 D7 (PA8), 50 Hz PWM; also to test-point row 9

Supplied from the HW-688 5 V rail through its current sensor, never from the Nucleo.

ACS712 Current Sensors (ACS712ELC-05B, 185 mV/A)

All three: VCC to the 5 V rail, GND to ground. OUT goes directly to an ADC pin (no divider; see Design Notes).

Sensor	Current path (IP+ → IP-)	OUT
Left motor	L298N OUT1 → IP+, IP- → left motor black wire	A3 (PB0)
Right motor	L298N OUT3 → IP+, IP- → right motor red wire	A2 (PA4)
Servo	5 V row → IP+, IP- → servo red wire	A4 (PC1)

Blinker LEDs

Four red LEDs, each driven by an STM32 GPIO through a 470 Ω resistor: pin → resistor → LED anode, cathode → ground. About 3 mA each.

LED	Header	Pin	Location	Test point
FL	D6	PB10	Front board, front-left corner	Row 10
FR	D10	PB6	Front board, front-right corner	Row 11
RL	A5	PC0	Rear board, rear-left corner	Row 12
RR	D3	PB3	Rear board, rear-right corner	Row 13

HW-688 Buck Converter

- VIN+ / VIN-: barrel-jack + / - (12 V)

- 5 V out: rear board 5 V rail
- GND out: main ground rail
- USB output and barrel-jack socket: unused

Barrel Jack (12 V input)

12 V wall adapter into a barrel-jack screw-terminal adapter. Two wires under each terminal: + to L298N 12V and HW-688 VIN+, – to L298N GND and HW-688 VIN–. Polarity verified with a multimeter before connecting.

Test-Point Board

Small breadboard at the chassis edge, one signal per row on the a–e side, labeled with tape.

Row	Signal	Source	Toggles on
1	GND	Main ground rail	-
2	GND	Main ground rail	-
3	UDP_RX	Pi GPIO23 (pin 16)	UDP packet received
4	CMD_TX	Pi GPIO24 (pin 18)	Command sent to STM32
5	CMD_RX	STM32 D9 (PC7)	Command received
6	PWM_SET	STM32 D13 (PA5)	New duty cycle written
7	PWM_OUT	STM32 D15 / L298N ENA	Motor PWM itself
8	DIR_A	STM32 D4 / L298N IN1	Direction / brake
9	SRV	STM32 D7 / servo signal	Servo PWM
10	FL	STM32 D6	Blinker
11	FR	STM32 D10	Blinker
12	RL	STM32 A5	Blinker
13	RR	STM32 D3	Blinker

5. Design Notes

Shared direction pins. IN1/IN3 share D4 and IN2/IN4 share D5 so the design fits on the Arduino headers. Both motors therefore always get the same direction and brake together, which is all this car needs. Because the motors are mirrored on the chassis, the left motor's leads are swapped so the same IN state drives both wheels forward. Dynamic brake is ENA/ENB high with IN1 = IN2 = low, which also gives DIR_A a clean falling edge when braking from forward.

High-side current sensing. Each ACS712 sits in series with the positive side of its load. The sensor's current path (IP+/IP–) is electrically isolated from its VCC/GND/OUT side, so IP+ sharing the 5 V net with VCC on the servo sensor is intended and is not a short. Because the

left motor's leads are swapped, its sensor reads with the opposite sign to the right sensor when driving forward.

No divider on the ACS712 outputs (temporary). $OUT = 2.5\text{ V} + 0.185\text{ V/A} \times I$, so it stays below 3.3 V for currents under about 4.3 A. The L298N is rated about 2 A per channel and the servo stalls well below that, so in normal use OUT stays roughly 2.1–2.9 V. A 1:2 divider (total $\geq 4.7\text{ k}\Omega$) should be added if resistors are available.

Encoders at 3.3 V. Encoders are powered from the Nucleo's 3.3 V so their A/B outputs never exceed the STM32's logic level.

CAN Integration Plan

For this lab, we have one thread that controls everything on one STM32. With the CAN integration and zone splitting, we have the following system:

Zone	Functions
Cockpit	UDP decode, buttons, blinker state machine, CAN TX on both buses, force-feedback UDP reply
Steering	<code>servo_set_angle</code> , steering scaling, FL/FR outputs, servo current turned into a force value
Drivetrain	encoders, PID, feed-forward table, brake and <code>brake_hold</code> , motor driver, stuck-wheel logic, RL/RR outputs, current sensing

** each STM32 zone will have its own self-test button and debounce

Each zone tracks CAN A and CAN B separately. If it hears nothing from any other zone on one bus for 3 heartbeat periods (60 ms) while the other bus is still active, it marks that bus as degraded. This information is reported in the heartbeat so that the cockpit can log it. The flag clears as soon as traffic resumes on that bus. A zone is declared failed only when its heartbeats are missing on both buses. If a zone hears nothing on either bus for 3 periods, it concludes it is isolated and enters its own fail-safe state.

The parser that currently follows the UART protocol i.e. ring buffer, resync, checksum, etc is replaced by the CAN protocol, which uses hardware ID, CRC, and framing. Lower ID means higher priority. Every message goes out on both buses with a seq, and receivers keep the first copy and drop the duplicate.

However, we may need to add two additional messages upon testing. Right now, the force feedback information is embedded in the heartbeats. Because the two messages have different timing requirements and different priorities within a system, we may need to split up this message. In addition, a potential issue could be with syncing the blinkers. We currently have one `k_timer` that sets all four pins. With two MCUs, there is a chance we fail to sync because of drift. If this happens, we will fix it by adding a blinker state machine to the cockpit that sends output levels in `WHEEL_CTRL`. Both zones receive the same broadcast frame, so edges differ minimally. The zones will discard their blink timers, and the cockpit sends an extra `WHEEL_CTRL` at each blink edge. A new `BLINKER` message will be added if this extra transmission of `WHEEL_CTRL` causes blink edge jitter.

ID	Name	Route	Period
0x1	WHEEL_CTRL	Cockpit → others	Every UDP update (<=50 ms) + every blinker edge
0x2	HB_COCKPIT	Cockpit → others	20 ms ± 10%
0x3	HB_STEERING	Steering → others	20 ms ± 10%
0x4	HB_DRIVETRAIN	Drivetrain → others	20 ms ± 10%
0x5	BLINKER_INFO	Cockpit → others	on every blink edge (500 ms turn, 250 ms hazard, both ± 10%)
0x6	FORCE_FEEDBACK	Steering → Cockpit	50 ms ± 10%

Team Defined Values Table

	Input	Valid Input Range	Output	Valid Output Range
Steering	Steering Wheel Angle	0 - 900 degrees	Angle on Steering Servo	0 - 180 degrees
Right Turn Threshold	Steering Wheel Angle	601 degrees	Angle on Steering Servo	53.4 degrees
Left Turn Threshold	Steering Wheel Angle	299 degrees	Angle on Steering Servo	26.6 degrees
Throttle-to-velocity	Throttle Value	-32768 to 32767	Velocity on Drive Motor:	0 - 230 rpm
Force Mapping	Current to Steering Servo	0 - 650 mA	Torque on Steering Wheel	-2.2 - 2.2 Nm

Task Table

Name	Period (ms)	Priority	Deadline (ms)
Brake	2	0x00	2
Throttle	2	0x01	2
Test button Local Fail	N/A	0x02	10
Steering	50	0x03	50

STM32 to PI Messages	20	0x03	20
Pi to STM32 Messages	50	0x03	50
STM Notice Link is Gone	N/A	0x04	150
Hazards	500	0x05	50
Turn signal	1000	0x05	100